UNIVERSITÀ DEGLI STUDI DI MILANO-BICOCCA

Dipartimento di Fisica «G. Occhialini»

Corso di Laurea Triennale



Sviluppo di un decoder basato su Reinforcement Learning per Quantum Error Correction

Relatore: Prof. Andrea Giachero

Correlatore: Dr. Danilo Labranca

Relatore esterno: Dr. Michele Grossi

Candidato:

Simone Pietro Razzetti

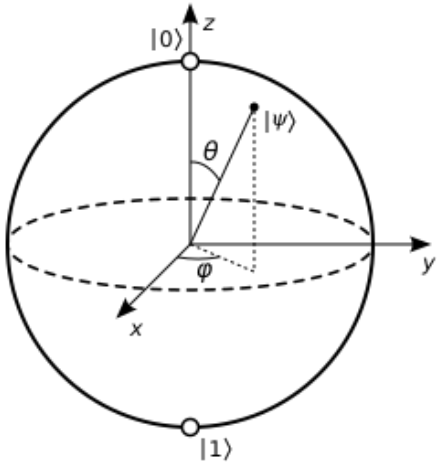
Matricola 898006

ANNO ACCADEMICO 2024/2025

Quantum Computing e Quantum Error Correction

- **Qubit**: sistema quantistico a 2 livelli come estensione del *bit* classico

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$$ o $$|\psi\rangle = \cos(\theta/2)|0\rangle + e^{i\phi}\sin(\theta/2)|1\rangle$$



Sfera di Bloch

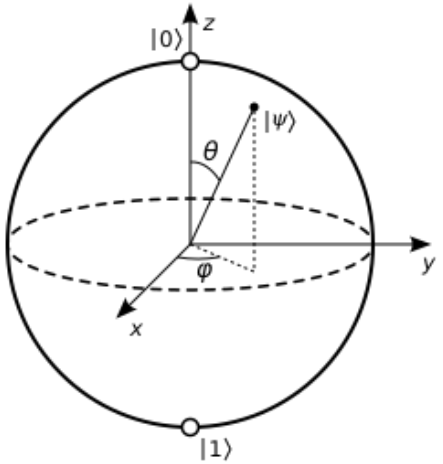
- **Potenzialità del calcolo quantistico:** sovrapposizione degli stati, ossia parallelismo quantistico, ed entanglement

- **Limite:** fragilità dovuta a fenomeni di decoerenza ed errori

Quantum Computing e Quantum Error Correction

- **Qubit**: sistema quantistico a 2 livelli come estensione del *bit* classico

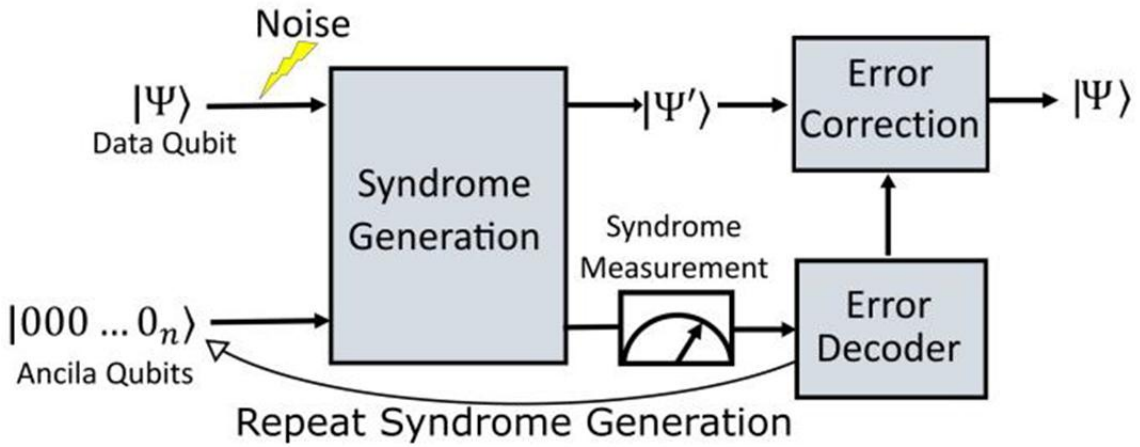
$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle$ o $|\psi\rangle = \cos(\theta/2)|0\rangle + e^{i\phi}\sin(\theta/2)|1\rangle$



Sfera di Bloch

- **Potenzialità del calcolo quantistico:** sovrapposizione degli stati, ossia parallelismo quantistico, ed entanglement

- **Limite:** fragilità dovuta a fenomeni di decoerenza ed errori



Quantum Error Correction (QEC)

- **Ridondanza**: un qubit logico attraverso più qubit fisici
- **Misura non distruttiva**: uso di opportuni operatori per rilevare in maniera indiretta l'errore senza collassare lo stato

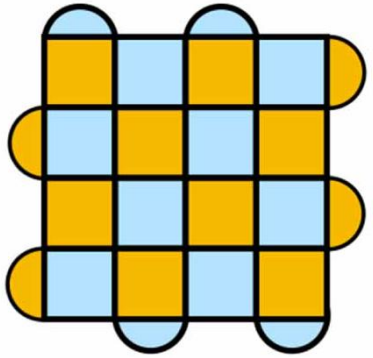
→ Codici a **Stabilizzatori** e misura della **Sindrome**

Surface Code e *Decoding problem*

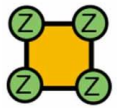
Codici topologici: informazione logica distribuita nello spazio e protetta dalle proprietà globali del sistema

Surface Code

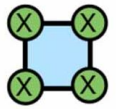
Struttura



Stabilizzatori

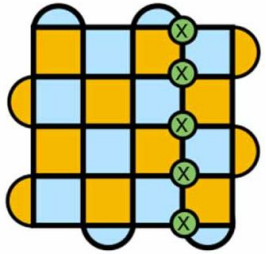


Sensibile a *bit-flip*
(Pauli X)

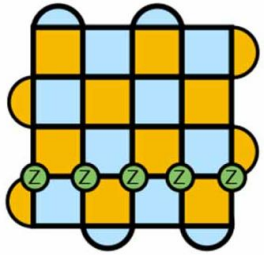


Sensibile a *phase-flip*
(Pauli Z)

Operatori logici

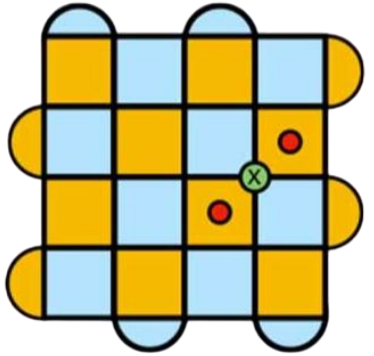


X_L



Z_L

Sindrome



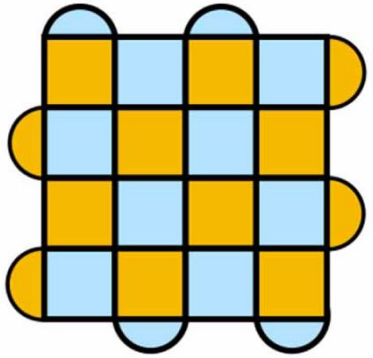
[*Google Quantum AI, Quantum error correction below the surface code threshold, Nature, 2025*]

Surface Code e *Decoding problem*

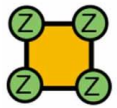
Codici topologici: informazione logica distribuita nello spazio e protetta dalle proprietà globali del sistema

Surface Code

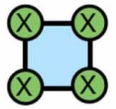
Struttura



Stabilizzatori

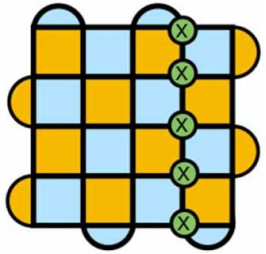


Sensibile a *bit-flip*
(Pauli X)

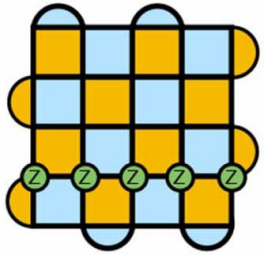


Sensibile a *phase-flip*
(Pauli Z)

Operatori logici

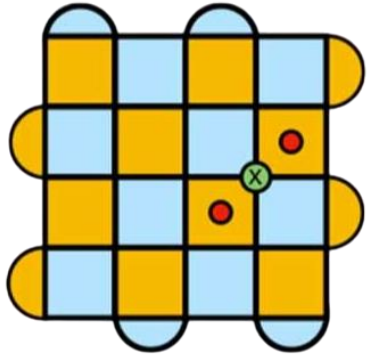


X_L



Z_L

Sindrome



Vantaggi:

- Soglia di errore (tolleranza al rumore) alta
- Scalabilità
- Compatibilità con le tecnologie hardware attuali

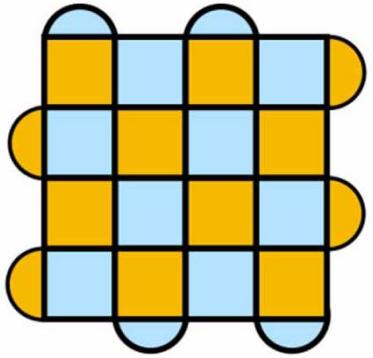
[*Google Quantum AI, Quantum error correction below the surface code threshold, Nature, 2025*]

Surface Code e *Decoding problem*

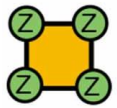
Codici topologici: informazione logica distribuita nello spazio e protetta dalle proprietà globali del sistema

Surface Code

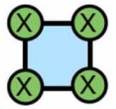
Struttura



Stabilizzatori

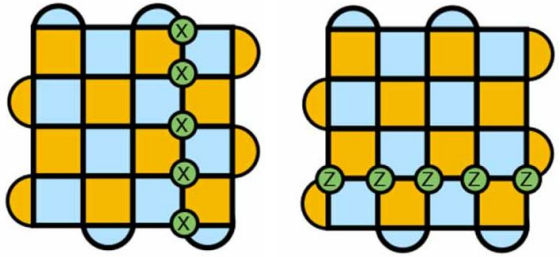


Sensibile a *bit-flip*
(Pauli X)



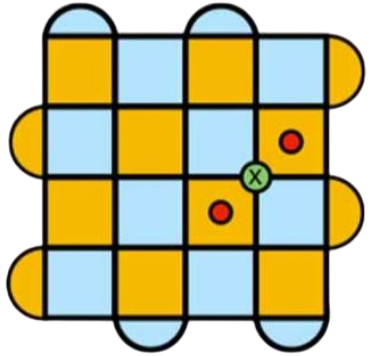
Sensibile a *phase-flip*
(Pauli Z)

Operatori logici



X_L Z_L

Sindrome



Vantaggi:

- Soglia di errore (tolleranza al rumore) alta
- Scalabilità
- Compatibilità con le tecnologie hardware attuali

Decoding problem

Data la misura di una sindrome, individuare la configurazione di errore più probabile e suggerire operazioni correttive.

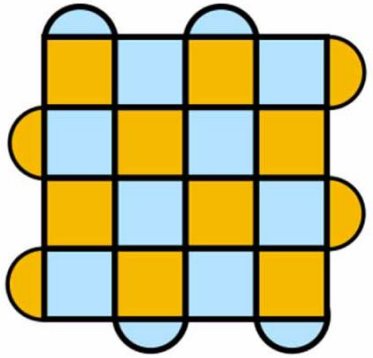
[*Google Quantum AI, Quantum error correction below the surface code threshold, Nature, 2025*]

Surface Code e *Decoding problem*

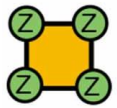
Codici topologici: informazione logica distribuita nello spazio e protetta dalle proprietà globali del sistema

Surface Code

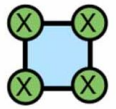
Struttura



Stabilizzatori

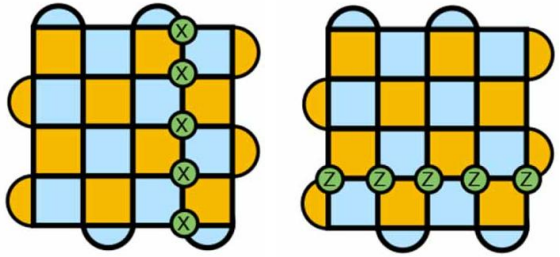


Sensibile a *bit-flip*
(Pauli X)



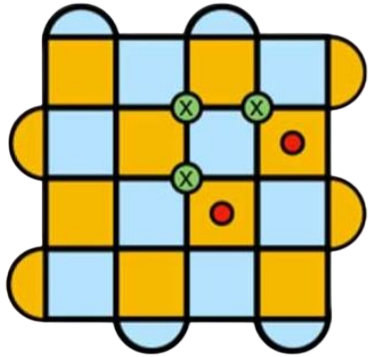
Sensibile a *phase-flip*
(Pauli Z)

Operatori logici



X_L Z_L

Sindrome



Vantaggi:

- Soglia di errore (tolleranza al rumore) alta
- Scalabilità
- Compatibilità con le tecnologie hardware attuali

Decoding problem

Data la misura di una sindrome, individuare la configurazione di errore più probabile e suggerire operazioni correttive.

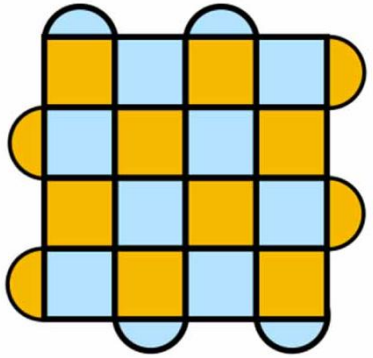
[*Google Quantum AI, Quantum error correction below the surface code threshold, Nature, 2025*]

Surface Code e *Decoding problem*

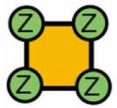
Codici topologici: informazione logica distribuita nello spazio e protetta dalle proprietà globali del sistema

Surface Code

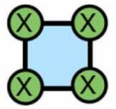
Struttura



Stabilizzatori

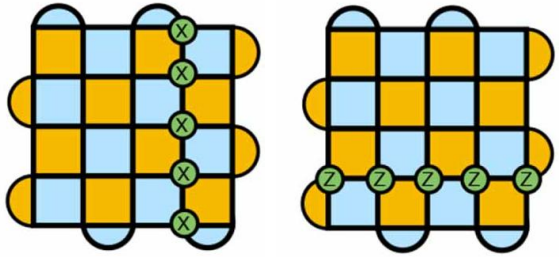


Sensibile a *bit-flip*
(Pauli X)



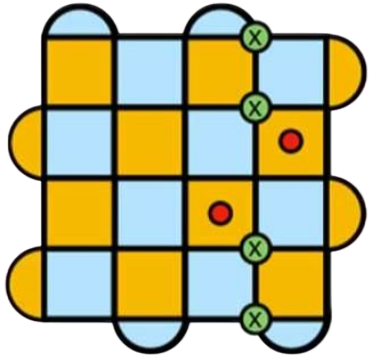
Sensibile a *phase-flip*
(Pauli Z)

Operatori logici



X_L Z_L

Sindrome



Vantaggi:

- Soglia di errore (tolleranza al rumore) alta
- Scalabilità
- Compatibilità con le tecnologie hardware attuali

Decoding problem

Data la misura di una sindrome, individuare la configurazione di errore più probabile e suggerire operazioni correttive.

[*Google Quantum AI, Quantum error correction below the surface code threshold, Nature, 2025*]

Problema, Obiettivi e Metodi

Decoding Problem

- Rumore complesso e non ideale
- Correzione *online*
- Scalabilità per codici grandi

Limiti dell'approccio classico

- Lento per codici grandi
- Adatto a modelli semplici
- Statico: no *faulty syndrome*

Metodi di Machine Learning

- Veloce *online* e più scalabile
- Generalizzabile
- Dinamico

Problema, Obiettivi e Metodi

Decoding Problem
- Rumore complesso e non ideale
- Correzione *online*
- Scalabilità per codici grandi

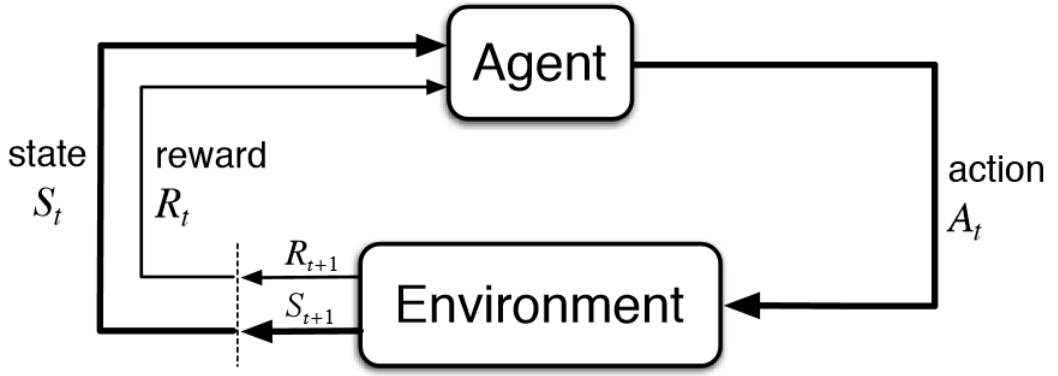
Limiti dell'approccio classico
- Lento per codici grandi
- Adatto a modelli semplici
- Statico: no *faulty syndrome*

Metodi di Machine Learning
- Veloce *online* e più scalabile
- Generalizzabile
- Dinamico

OBIETTIVO I: Sviluppo di un decoder basato sul paradigma di **Reinforcement Learning (RL)** per decodifica di sindromi relative a Surface Code affette da errori di misura (*faulty)*

Processo Decisionale di Markov (MDP)

L'**agente** (*decoder*) impara a prendere **decisioni sequenziali ottimali** (*correzioni*) interagendo con un **ambiente** (*Surface Code*) per massimizzare una **ricompensa** (*riduzione dell'errore logico*).



Problema, Obiettivi e Metodi

Decoding Problem
- Rumore complesso e non ideale
- Correzione *online*
- Scalabilità per codici grandi

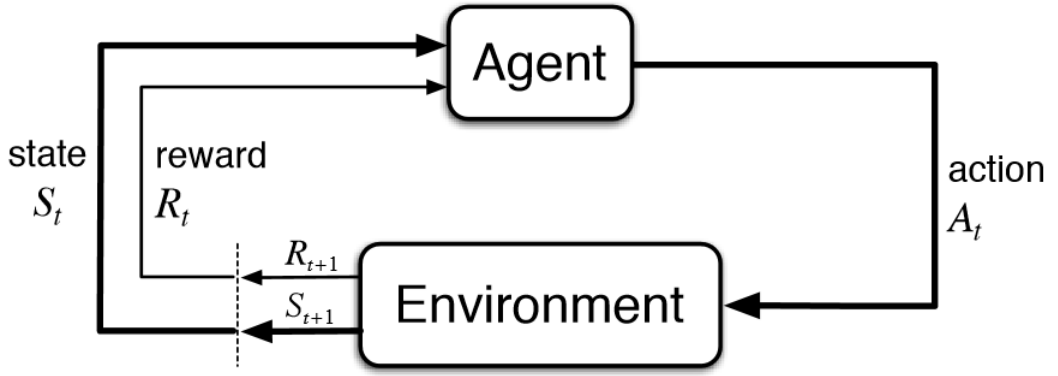
Limiti dell'approccio classico
- Lento per codici grandi
- Adatto a modelli semplici
- Statico: no *faulty syndrome*

Metodi di Machine Learning
- Veloce *online* e più scalabile
- Generalizzabile
- Dinamico

OBIETTIVO I: Sviluppo di un decoder basato sul paradigma di **Reinforcement Learning (RL)** per decodifica di sindromi relative a Surface Code affette da errori di misura (*faulty)*

Processo Decisionale di Markov (MDP)

L'**agente** (*decoder*) impara a prendere **decisioni sequenziali ottimali** (*correzioni*) interagendo con un **ambiente** (*Surface Code*) per massimizzare una **ricompensa** (*riduzione dell'errore logico*).



Limite: decoder presto inefficace in ambienti variabili

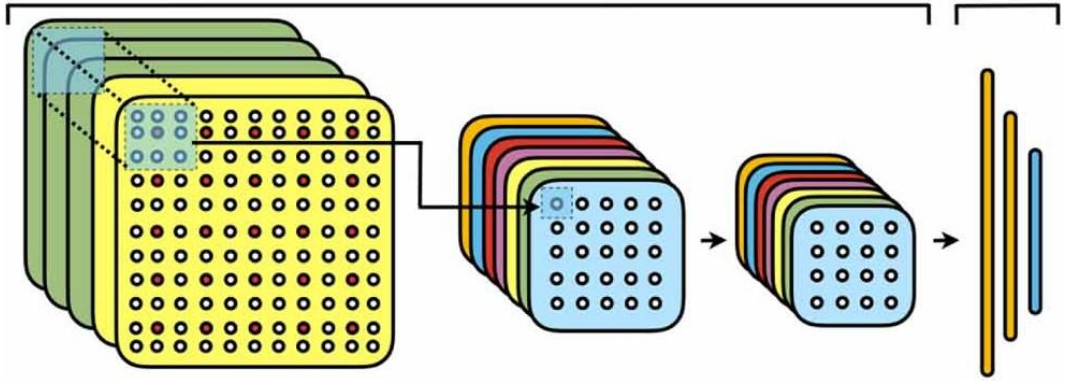
OBIETTIVO II: Studio degli effetti di **Continual Learning** per l'adattabilità e riduzione dei costi di ri-addestramento.

Deep RL: Double Deep Q-Network (DDQN)

Obiettivo dell'addestramento RL: trovare la *policy* ottimale → Stima della **funzione valore** $Q(s, a)$

Rete Neurale Convoluzionale (CNN)

Input:
Sindrome *dinamica*



Output:
Stima del valore di ciascun azione (correzione possibile)

Double Deep Q-Network:

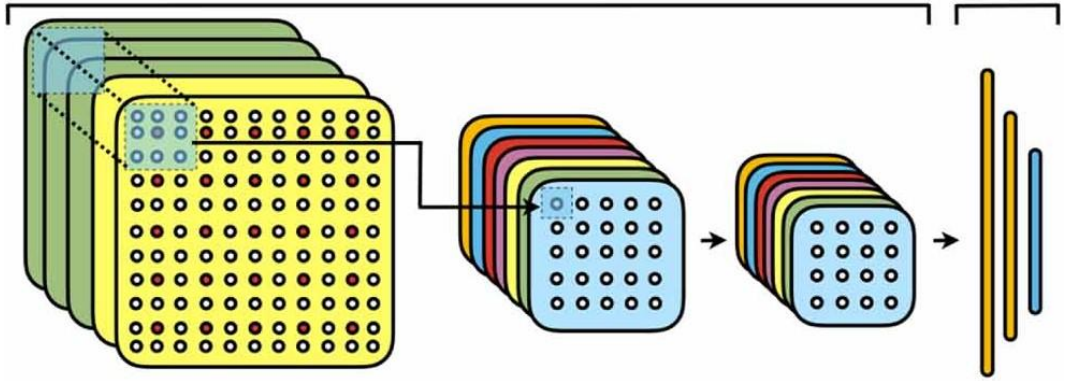
- Una **Rete Neurale Policy** approssima $Q(s, a)$ per ogni coppia stato-azione (*sindrome-correzione*)

Deep RL: Double Deep Q-Network (DDQN)

Obiettivo dell'addestramento RL: trovare la *policy* ottimale → Stima della **funzione valore** $Q(s,a)$

Input:
Sindrome *dinamica*

Rete Neurale Convoluzionale (CNN)



Output:
Stima del valore di ciascun azione (correzione possibile)

Double Deep Q-Network:

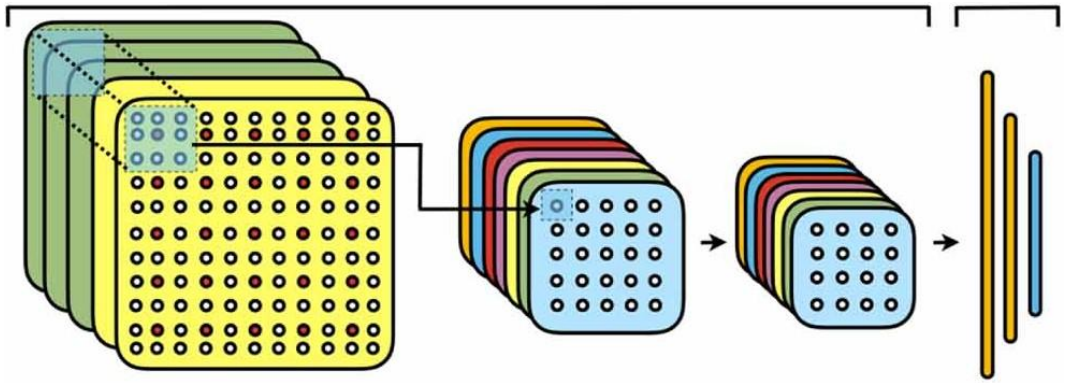
- Una **Rete Neurale Policy** approssima $Q(s,a)$ per ogni coppia stato-azione (*sindrome-correzione*)
- Una **Rete Neurale Target** per calcola il valore obiettivo ad ogni ottimizzazione dei pesi
- L'agente accumula esperienza nel **Replay Buffer** interagendo direttamente in maniera sequenziale con l'ambiente

Deep RL: Double Deep Q-Network (DDQN)

Obiettivo dell'addestramento RL: trovare la *policy* ottimale → Stima della **funzione valore** $Q(s, a)$

Rete Neurale Convoluzionale (CNN)

Input:
Sindrome *dinamica*



Output:
Stima del valore di ciascun azione (correzione possibile)

Double Deep Q-Network:

- Una **Rete Neurale Policy** approssima $Q(s, a)$ per ogni coppia stato-azione (*sindrome-correzione*)
- Una **Rete Neurale Target** per calcola il valore obiettivo ad ogni ottimizzazione dei pesi
- L'agente accumula esperienza nel **Replay Buffer** interagendo direttamente in maniera sequenziale con l'ambiente
- ***Double:*** la rete policy *seleziona* l'azione ottimale, la rete target *calcola* il valore dell'obiettivo → Riduce il rischio di sovrastima di $Q(s, a)$

Continual Learning: Probabilistic Policy Reuse (PPR)

Continual Learning: apprendimento basato sul riutilizzo delle conoscenze acquisite per agevolare un nuovo addestramento

Algoritmi *off-policy*
Strategia di selezione dell'azione:

Continual Learning: Probabilistic Policy Reuse (PPR)

Continual Learning: apprendimento basato sul riutilizzo delle conoscenze acquisite per agevolare un nuovo addestramento

Algoritmi *off-policy*
Strategia di selezione dell'azione:

ϵ-greedy

- Esplorazione casuale con prob. ϵ
- Sfruttamento di π_{new} con prob. $(1-\epsilon)$

→ **DDQN**

Continual Learning: Probabilistic Policy Reuse (PPR)

Continual Learning: apprendimento basato sul riutilizzo delle conoscenze acquisite per agevolare un nuovo addestramento

Algoritmi *off-policy*
Strategia di selezione dell'azione:

ϵ-greedy

- Esplorazione casuale con prob. ϵ
- Sfruttamento di π_{new} con prob. $(1-\epsilon)$

→ **DDQN**

π-reuse

- Sfruttamento di π_{past} con prob. ψ
- Strategia ϵ-*greedy* su π_{new} con prob. $(1-\psi)$

→ **DDQN + PPR**

Continual Learning: Probabilistic Policy Reuse (PPR)

Continual Learning: apprendimento basato sul riutilizzo delle conoscenze acquisite per agevolare un nuovo addestramento

Algoritmi *off-policy*
Strategia di selezione dell'azione:

ϵ-greedy	Esplorazione casuale Sfruttamento di π_{new}	con prob. ϵ con prob. $(1-\epsilon)$	**DDQN**
π-reuse	Sfruttamento di π_{past} Strategia *ϵ-greedy* su π_{new}	con prob. ψ con prob. $(1-\psi)$	**DDQN + PPR**

Probabilistic Policy Reuse: Data una libreria L di *policy* pre-addestrate in ambienti simili, per ogni episodio di interazione viene selezionata una strategia π_{past} per mettere in pratica *π-reuse*

Distribuzione Softmax $$P(\pi_j) = \frac{e^{\tau W_j}}{\sum_{p=0}^{n} e^{\tau W_p}}$$

- τ parametro di *temperatura* e δ_τ incremento per ogni ep.
- W_j ricompensa media della politica π_j

Setup sperimentale e simulazioni

Il progetto è stato interamente sviluppato in **Python**, ispirato ai lavori di Sweke et al. (2020) e Matekole et al. (2022)

Setup sperimentale e simulazioni

Il progetto è stato interamente sviluppato in **Python**, ispirato ai lavori di Sweke et al. (2020) e Matekole et al. (2022)

Agente

- Implementato attraverso opportune classi sviluppate con la libreria **PyTorch**
- Comprende gli elementi necessari a **DDQN** e **PPR**: Reti Neurali, Replay Buffer, Ottimizzatore

Setup sperimentale e simulazioni

Il progetto è stato interamente sviluppato in **Python**, ispirato ai lavori di Sweke et al. (2020) e Matekole et al. (2022)

Agente

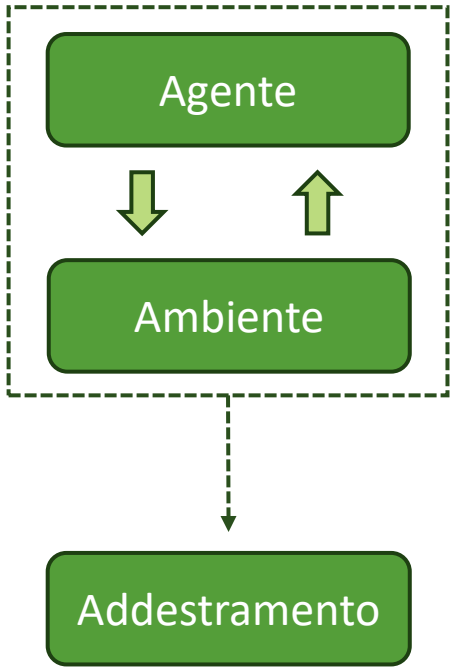
- Implementato attraverso opportune classi sviluppate con la libreria **PyTorch**
- Comprende gli elementi necessari a **DDQN** e **PPR**: Reti Neurali, Replay Buffer, Ottimizzatore

Ambiente

- Tratto dal repository di Sweke et al. (2020), aggiornato alla libreria **Gymnasium**
- Simula *volumi di sindrome dinamici* relativi a Surface Code 5x5 soggetto a un rumore con p_{phys} e p_{meas}
- Ambienti simulati: solo errore ***bit-flip*** con tasso $p_{err} = p_{phys} = p_{meas}$ crescente.

Setup sperimentale e simulazioni

Il progetto è stato interamente sviluppato in **Python**, ispirato ai lavori di Sweke et al. (2020) e Matekole et al. (2022)



- Implementato attraverso opportune classi sviluppate con la libreria **PyTorch**
- Comprende gli elementi necessari a **DDQN** e **PPR**: Reti Neurali, Replay Buffer, Ottimizzatore

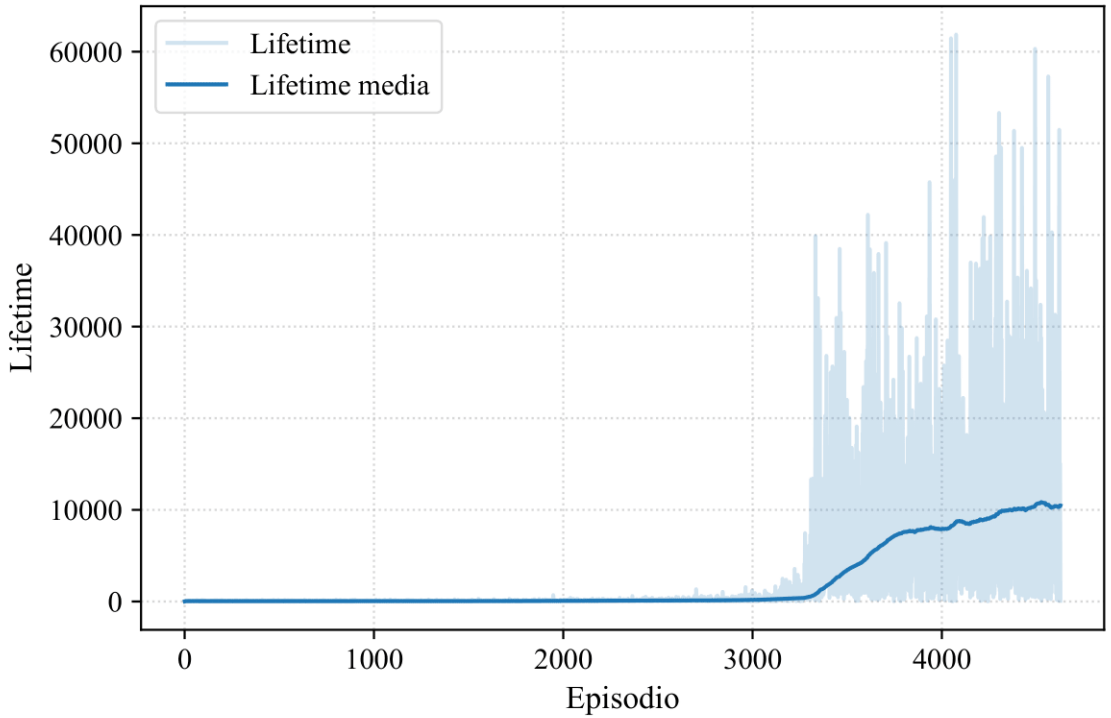
- Tratto dal repository di Sweke et al. (2020), aggiornato alla libreria **Gymnasium**
- Simula *volumi di sindrome dinamici* relativi a Surface Code 5x5 soggetto a un rumore con p_{phys} e p_{meas}
- Ambienti simulati: solo errore ***bit-flip*** con tasso $p_{err} = p_{phys} = p_{meas}$ crescente.

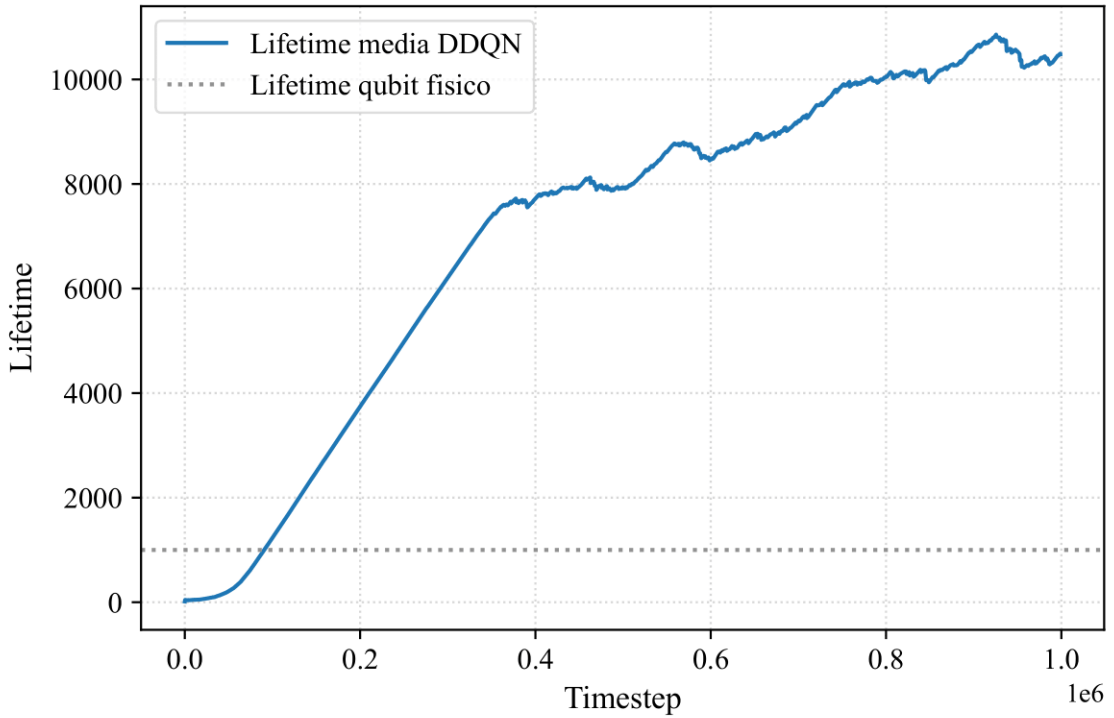
- Funzioni distinte **DDQN** e **DDQN + PPR**
- Gli step di interazione sono articolati su **episodi**, durante i quali si alternano ***misure*** **della sindrome** e **azioni correttive**, fino al raggiungimento di una condizione terminale (***errore logico***).
- **Gestione dinamica** *(semplice)* degli iperparametri **ε** e **ψ**

Logical lifetime Ciascun ciclo misura-correzioni rappresenta un istante di tempo discreto: il numero di sindromi raccolte in un episodio prima di una condizione terminale viene estratto come *lifetime* del sistema.

Risultati sperimentali: DDQN vs DDQN + PPR

Ambiente $p_{err} = 0.001$

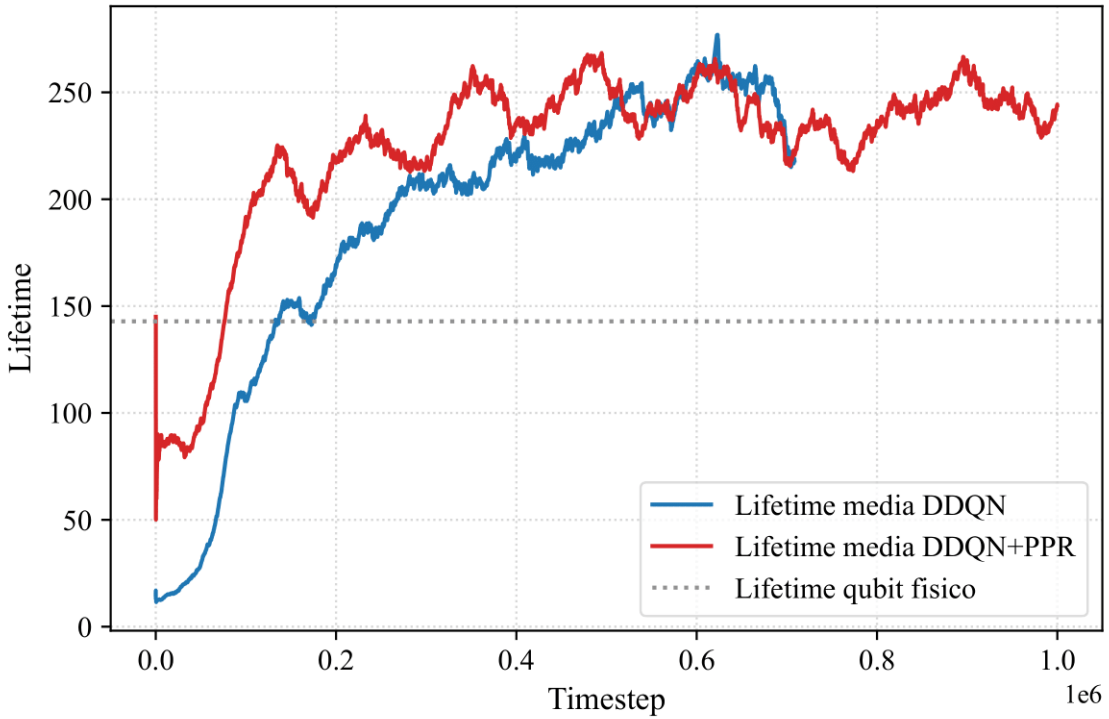




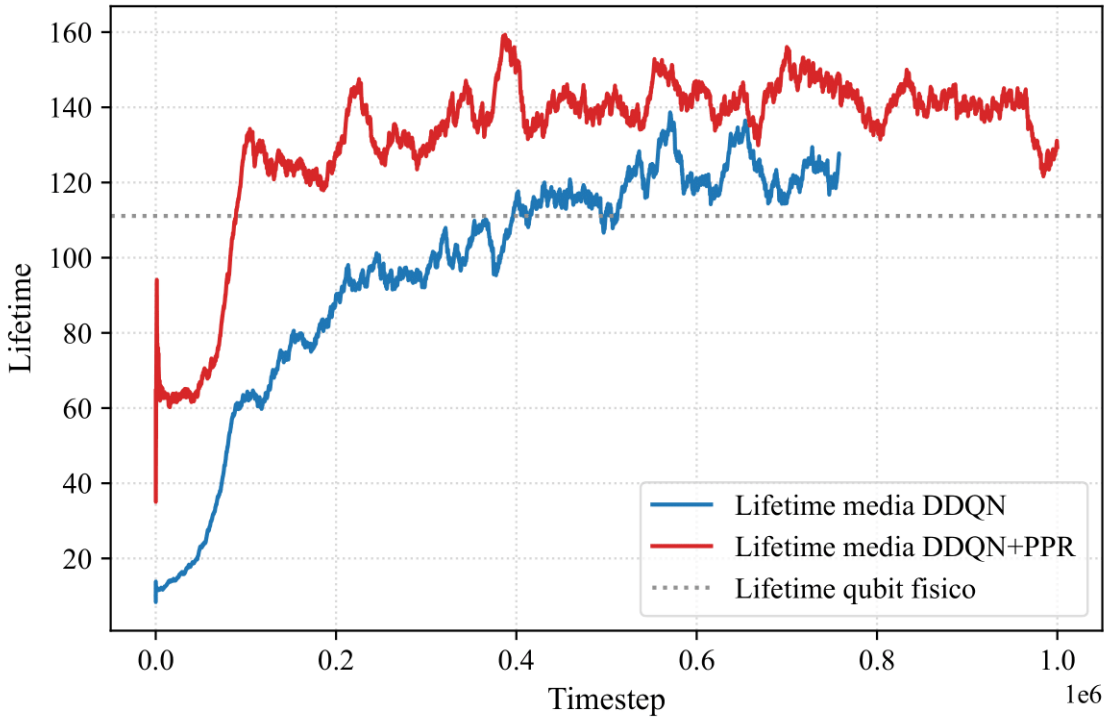
La ***Logical Lifetime*** è stata scelta come metrica principale, in sostituzione della Ricompensa standard RL, per fornire una valutazione *fisica* e diretta dell'efficacia del decoder.

Risultati sperimentali: DDQN vs DDQN + PPR (cont.)

Ambiente $p_{err} = 0.007$

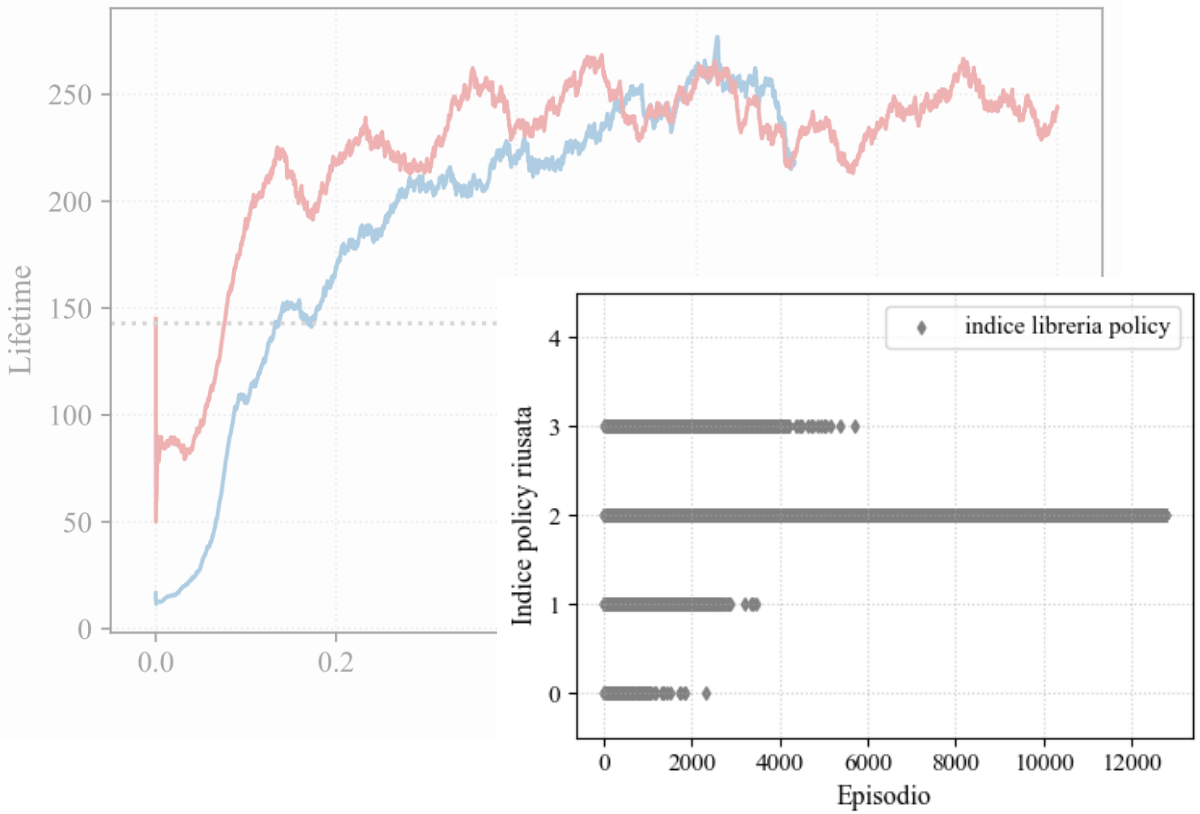


Ambiente $p_{err} = 0.009$

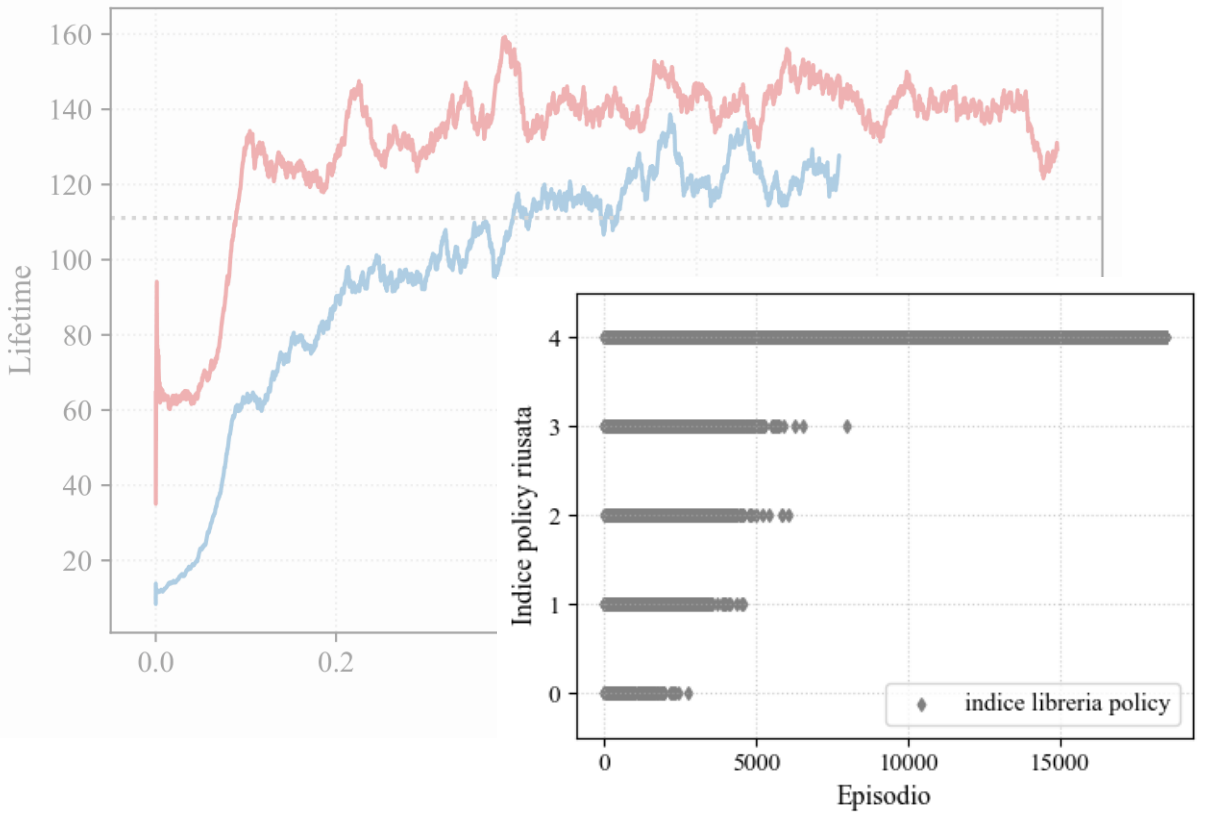


Risultati sperimentali: DDQN vs DDQN + PPR (cont.)

Ambiente $p_{err} = 0.007$

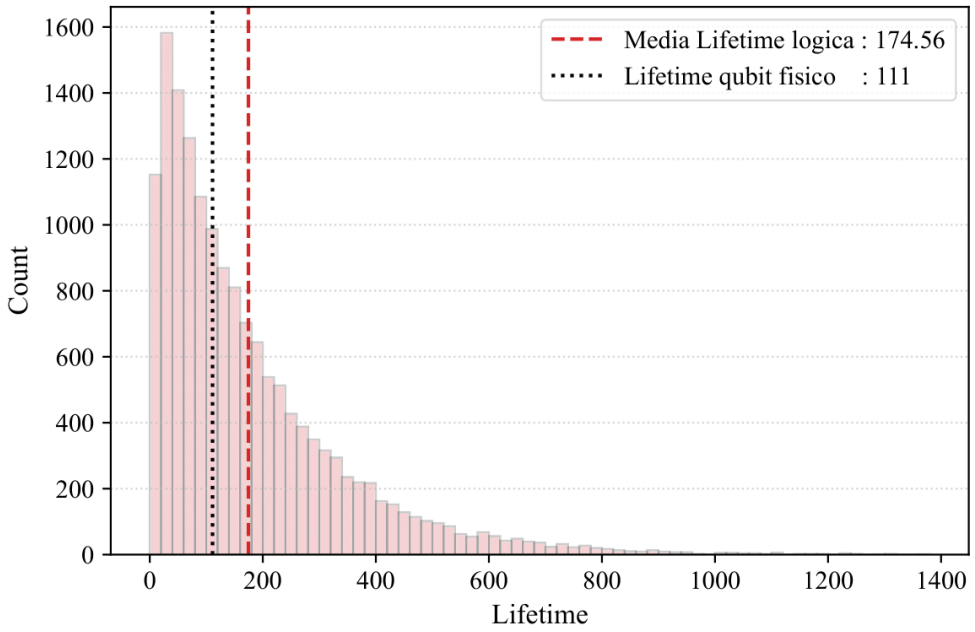


Ambiente $p_{err} = 0.009$



Risultati sperimentali: Analisi *Logical Lifetime*

- **Natura poissoniana** degli errori logici: il decoder migliora la *Logical Lifetime*, ma non modifica la forma della distribuzione

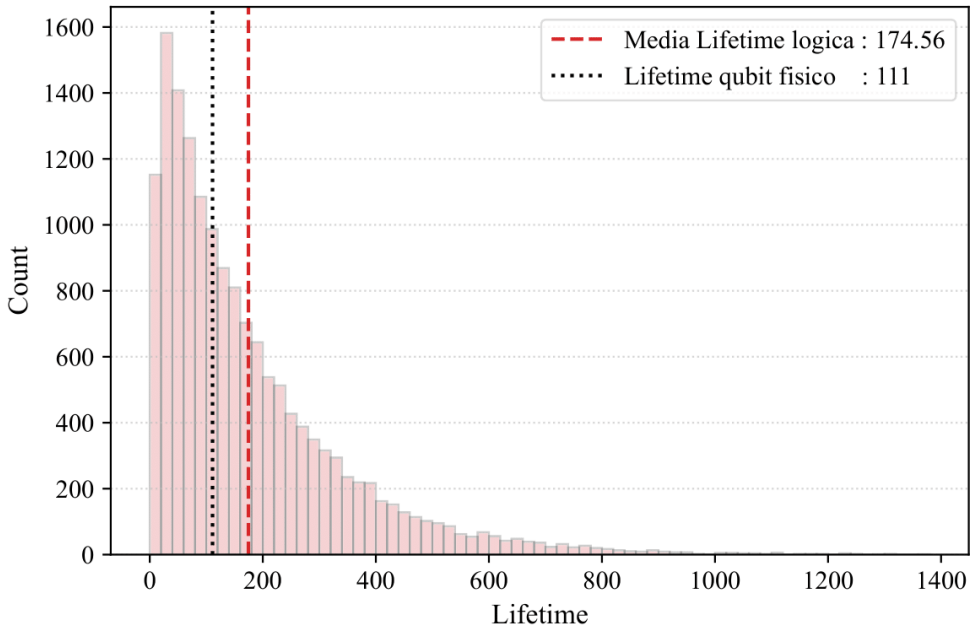


Esempio: Distribuzione *Logical Lifetime* del test policy DDQN + PPR in amb. p_{err} = 0.009

Risultati sperimentali: Analisi *Logical Lifetime*

- **Natura poissoniana** degli errori logici: il decoder migliora la *Logical Lifetime*, ma non modifica la forma della distribuzione
- **Stime concrete**: *Logical Lifetime* in condizioni *faulty*

Ambiente p_{err}	Qubit fisico *Lifetime*	DDQN *Logical Lifetime*	DDQN + PPR *Logical Lifetime*
0.001	1000	16723	/
0.003	333	1794	2066
0.005	200	653	611
0.007	142	340	321
0.009	111	163	175

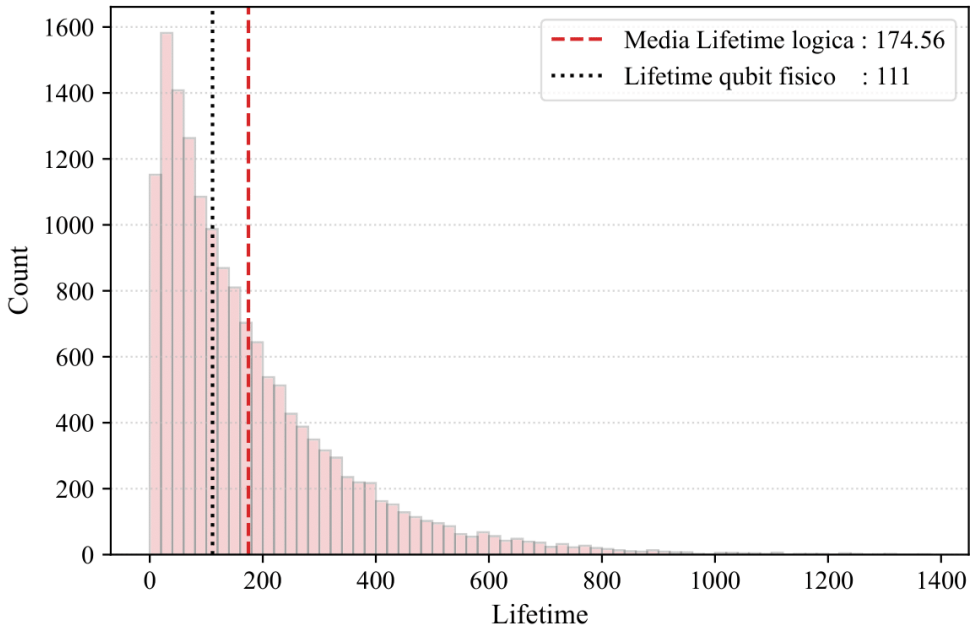


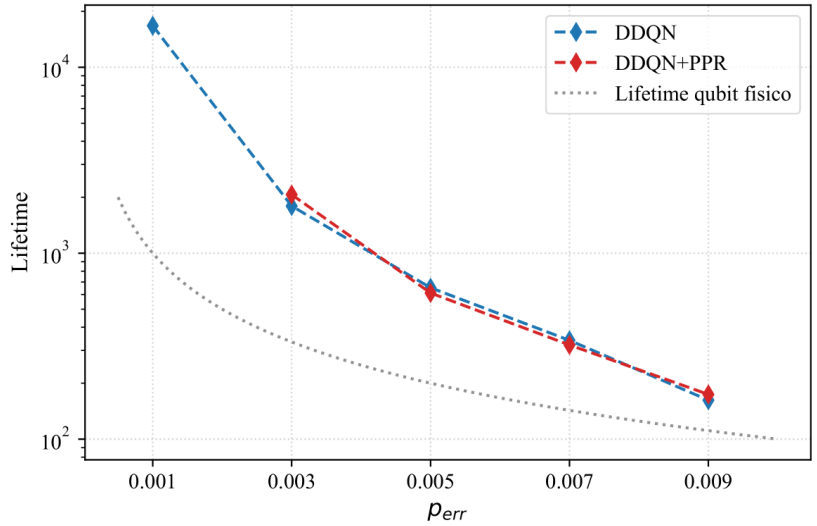
Esempio: Distribuzione *Logical Lifetime* del test policy DDQN + PPR in amb. p_{err} = 0.009

Risultati sperimentali: Analisi *Logical Lifetime*

- **Natura poissoniana** degli errori logici: il decoder migliora la *Logical Lifetime*, ma non modifica la forma della distribuzione
- **Stime concrete**: *Logical Lifetime* in condizioni *faulty*
- **Limite pratico**: Impatto della crescente ambiguità della sindrome

Ambiente p_{err}	Qubit fisico *Lifetime*	DDQN *Logical Lifetime*	DDQN + PPR *Logical Lifetime*
0.001	1000	16723	/
0.003	333	1794	2066
0.005	200	653	611
0.007	142	340	321
0.009	111	163	175





Esempio: Distribuzione *Logical Lifetime* del test policy DDQN + PPR in amb. p_{err} = 0.009

Conclusioni e prospettive future

Dati sperimentali

Prospettive future

[*Sivak et al., Reinforcement Learning Control of Quantum Error Correction (Google Quantum AI, 2025)*]

Conclusioni e prospettive future

Dati sperimentali

- **Potenzialità decoder:** aumento significativo della *Logical Lifetime*, contestualizzato alla complessità di ogni ambiente.
- **Efficienza PPR:** convergenza più rapida del 20-30% rispetto a DDQN tradizionale.

➡ I dati ottenuti rappresentano un primo risultato promettente che conferma la validità dei metodi proposti

Prospettive future

[*Sivak et al., Reinforcement Learning Control of Quantum Error Correction (Google Quantum AI, 2025)*]

Conclusioni e prospettive future

Dati sperimentali

- **Potenzialità decoder:** aumento significativo della *Logical Lifetime*, contestualizzato alla complessità di ogni ambiente.
- **Efficienza PPR:** convergenza più rapida del 20-30% rispetto a DDQN tradizionale.

➡ I dati ottenuti rappresentano un primo risultato promettente che conferma la validità dei metodi proposti

Prospettive future

- **Ottimizzazione HPC**: impiego di risorse *High Performance Computing* per ottimizzazione più approfondita dei parametri di training

[*Sivak et al., Reinforcement Learning Control of Quantum Error Correction (Google Quantum AI, 2025)*]

Conclusioni e prospettive future

Dati sperimentali

- **Potenzialità decoder:** aumento significativo della *Logical Lifetime*, contestualizzato alla complessità di ogni ambiente.
- **Efficienza PPR:** convergenza più rapida del 20-30% rispetto a DDQN tradizionale.

➡ I dati ottenuti rappresentano un primo risultato promettente che conferma la validità dei metodi proposti

Prospettive future

- **Ottimizzazione HPC**: impiego di risorse *High Performance Computing* per ottimizzazione più approfondita dei parametri di training
- **Gestione adattiva di ε e ψ:** regolazione dinamica basata sul contesto ambientale e sulla somiglianza del rumore per ottimizzare l'adattamento

[*Sivak et al., Reinforcement Learning Control of Quantum Error Correction (Google Quantum AI, 2025)*]

Conclusioni e prospettive future

Dati sperimentali

- **Potenzialità decoder:** aumento significativo della *Logical Lifetime*, contestualizzato alla complessità di ogni ambiente.
- **Efficienza PPR:** convergenza più rapida del 20-30% rispetto a DDQN tradizionale.

➡ I dati ottenuti rappresentano un primo risultato promettente che conferma la validità dei metodi proposti

Prospettive future

- **Ottimizzazione HPC**: impiego di risorse *High Performance Computing* per ottimizzazione più approfondita dei parametri di training
- **Gestione adattiva di ε e ψ:** regolazione dinamica basata sul contesto ambientale e sulla somiglianza del rumore per ottimizzare l'adattamento
- ***(Probabilistic) Policy Selection:*** Meccanismo di selezione dinamica della policy per ambienti soggetti a rumore variabile (*drift*).

[*Sivak et al., Reinforcement Learning Control of Quantum Error Correction (Google Quantum AI, 2025)*]

Conclusioni e prospettive future

Dati sperimentali

- **Potenzialità decoder:** aumento significativo della *Logical Lifetime*, contestualizzato alla complessità di ogni ambiente.
- **Efficienza PPR:** convergenza più rapida del 20-30% rispetto a DDQN tradizionale.

➡ I dati ottenuti rappresentano un primo risultato promettente che conferma la validità dei metodi proposti

Prospettive future

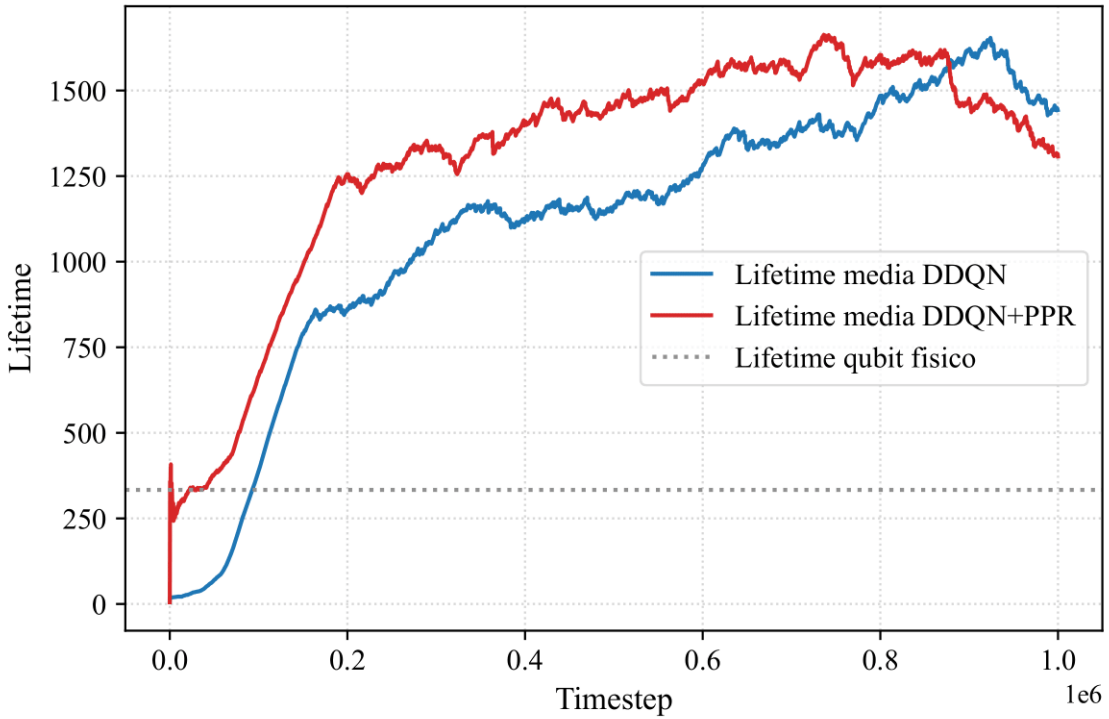
- **Ottimizzazione HPC**: impiego di risorse *High Performance Computing* per ottimizzazione più approfondita dei parametri di training
- **Gestione adattiva di ε e ψ:** regolazione dinamica basata sul contesto ambientale e sulla somiglianza del rumore per ottimizzare l'adattamento
- ***(Probabilistic) Policy Selection:*** Meccanismo di selezione dinamica della policy per ambienti soggetti a rumore variabile (*drift*).
- **Qubit reali:** utilizzo di *qubit superconduttivi* per simulazioni avanzate e implementazione sperimentale

[*Sivak et al., Reinforcement Learning Control of Quantum Error Correction (Google Quantum AI, 2025)*]

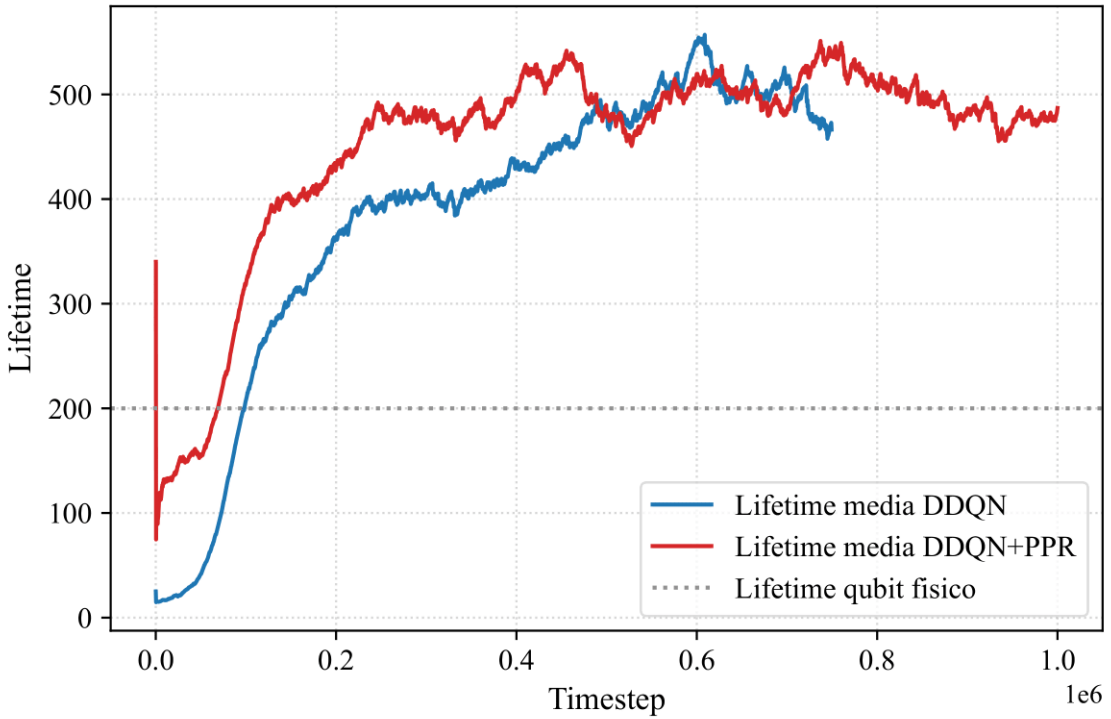
Grazie per l’attenzione!

Risultati sperimentali: DDQN vs DDQN + PPR

Ambiente $p_{err} = 0.003$

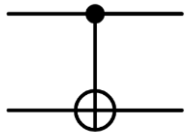


Ambiente $p_{err} = 0.005$



Quantum gate, diagrammi di circuito e modelli di rumore

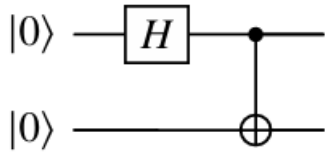
Quantum Gate

- ***X-gate***  $\mathbf{X} = \sigma_x = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ $X|0\rangle = |1\rangle, \quad X|1\rangle = |0\rangle$
- ***Z-gate***  $\mathbf{Z} = \sigma_z = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}$ $Z|0\rangle = |0\rangle, \quad Z|1\rangle = -|1\rangle$
- ***Hadamard-gate***  $\mathbf{H} = \frac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$ $H|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |+\rangle$
- ***Controlled-X gate***  $\mathbf{CX} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$ $CX|a\rangle|b\rangle = |a\rangle|a \oplus b\rangle$

Modelli di rumore

- *Bit-flip Noise* 
- *Phase-flip Noise* 
- *Depolarizing Noise* 

Circuiti Quantistici



Stato di Bell $|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$

Stabilizer states

Dati *n* qubit fisici e *k* qubit logici:

- ***Stabilizer Group S*:** sottogruppo abeliano del gruppo di Pauli P_n , generato da $n-k$ operatori S_i indipendenti

$$S = \langle S_i \rangle$$

- ***Stabilizer State*** $|\psi\rangle$***:*** unico autostato di tutti i generatori con autovalore +1

$$S_i|\psi\rangle = +1 \cdot |\psi\rangle$$

In **Quantum Error Correction:**

- I Codici a Stabilizzatori (come Surface Code) sono definiti da un *Stabilizer Group*
- Lo stato logico del codice è definito da *Stabilizer States*
- La sindrome si basa sulla misurazione dell'autovalore degli stabilizzatori

⇨ Lo stato di un Codice a Stabilizzatori può essere descritto in modo compatto a partire da $O(n)$ operatori (invece che 2^nelementi), pertanto può essere simulato efficientemente su un computer classico (*Teorema di Gottesman-Knill*)

Misura indiretta degli stabilizzatori

«Quantum» repetition code

Stato logico

$$|\psi\rangle = \alpha|0\rangle_L + \beta|1\rangle_L = \alpha|000\rangle + \beta|111\rangle$$

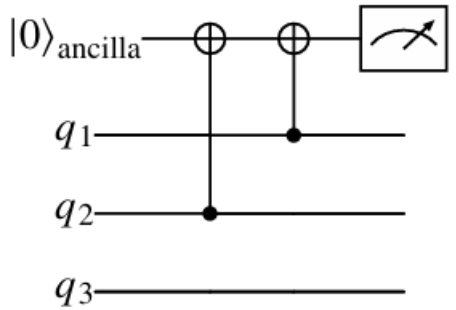
Stabilizzatori (Bit-flip)

$Z_1Z_2 \quad Z_2Z_3$

Sindrome

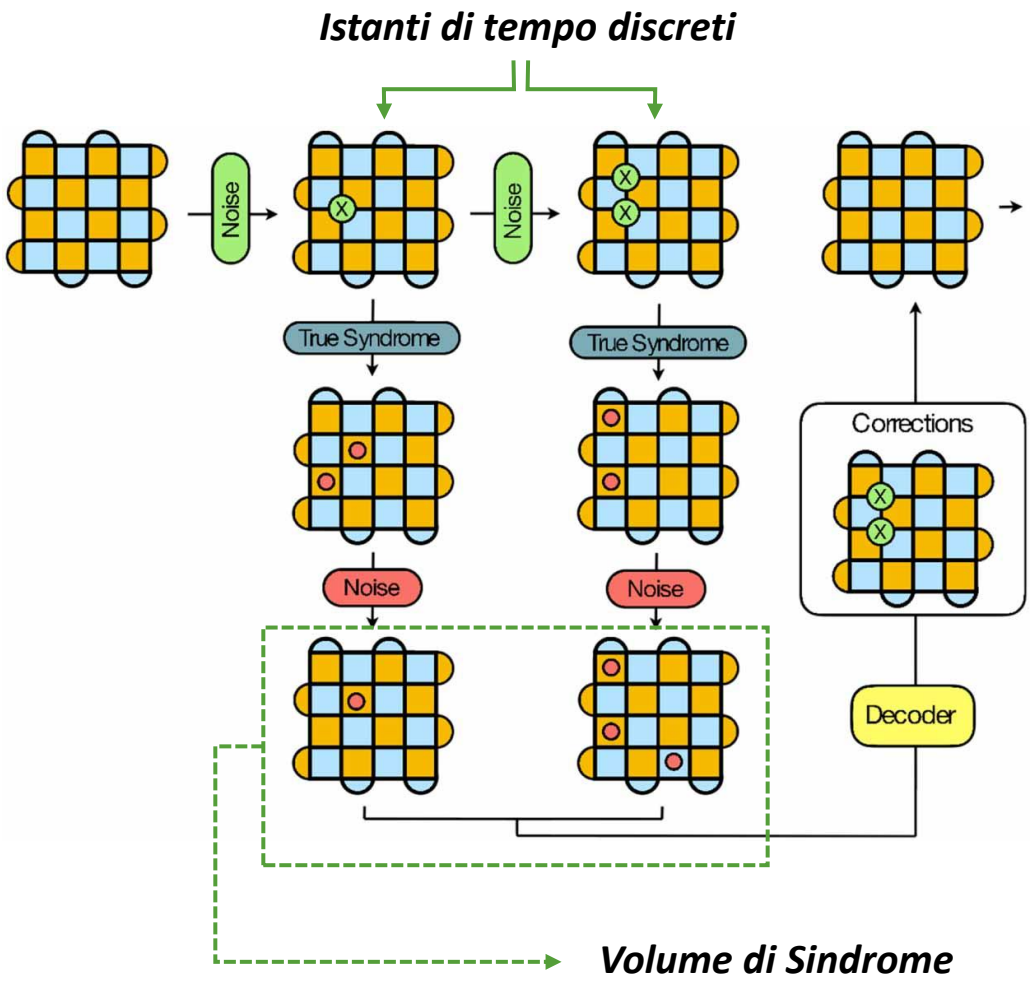
Errore	Stato corrotto	Z_1Z_2	Z_2Z_3
Nessuno	$\alpha\|000\rangle + \beta\|111\rangle$	$+1$	$+1$
X_1	$\alpha\|100\rangle + \beta\|011\rangle$	-1	$+1$
X_2	$\alpha\|010\rangle + \beta\|101\rangle$	-1	-1
X_3	$\alpha\|001\rangle + \beta\|110\rangle$	$+1$	-1

Misura indiretta di Z_1Z_2



L'informazione sull'errore (la parità Z_1Z_2) viene trasferita al qubit ancilla tramite operatori *CX*, la cui successiva misura ne rivela l'esito senza collassare lo stato logico.

Processo di misura sequenziale e *Faulty Syndrome Volume*



- L'ambiente simula sequenzialmente la presenza di errori fisici nel codice, da cui viene estratta la *true syndrome*
- A partire da ciascuna *true syndrome*, vengono simulati errori nei processi di misura ottenendo *faulty syndrome*
- L'insieme delle matrici di *faulty syndrome* costituisce il **Volume di Sindrome**

Ciclo di interazione MDP

- Gli step di interazione sono articolati su **episodi**, durante i quali si alternano ***misure*** **della sindrome** e ***azioni*** **correttive**, fino al raggiungimento di una condizione terminale (***errore logico***)
- **Apprendimento non convenzionale:** non è necessario ripristinare lo stato logico prima di una nuova misura
- Il numero di sindromi raccolte in un episodio prima di una condizione terminale viene estratto come ***lifetime*** del sistema

